Multimodal Semantic Image Search

Arian, Eshan, Kailash

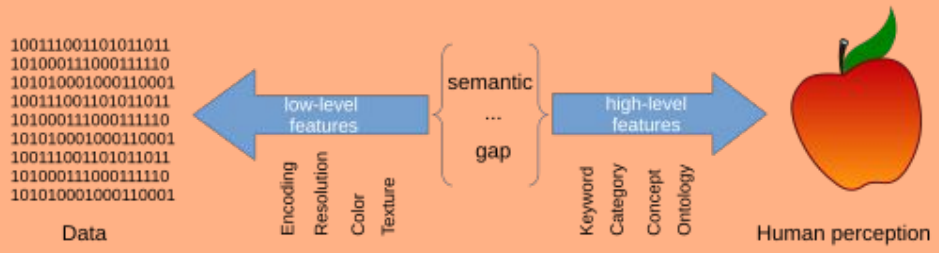
The Problem

Traditional Approach Limitations:

- Relies on human-generated metadata (tags, filenames).
- Unscalable for massive, unstructured datasets.
- Computer vision traditionally looks for pixel frequencies, not meaning.

The Neural Network Solution:

- Discard manual tagging entirely.
- Project both text and images into a shared mathematical dimension.



Architecture (CLIP)

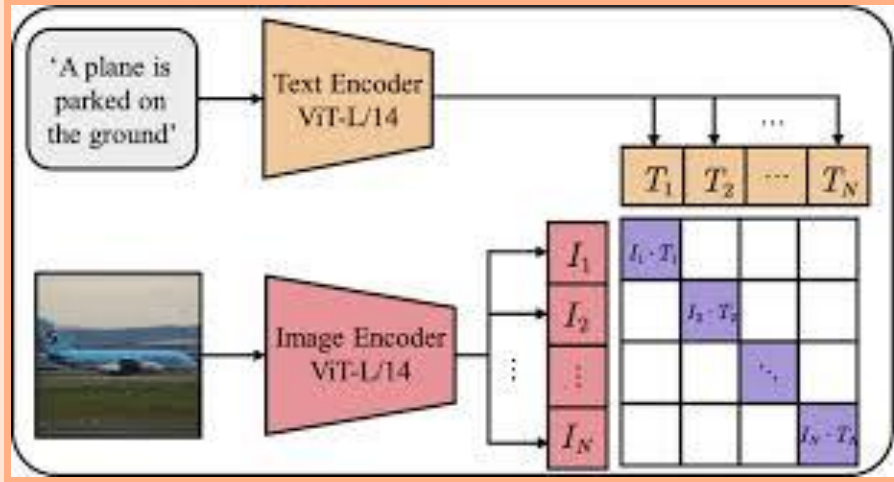
Model: CLIP (Contrastive Language-Image Pretraining).

Dual-Encoder Design:

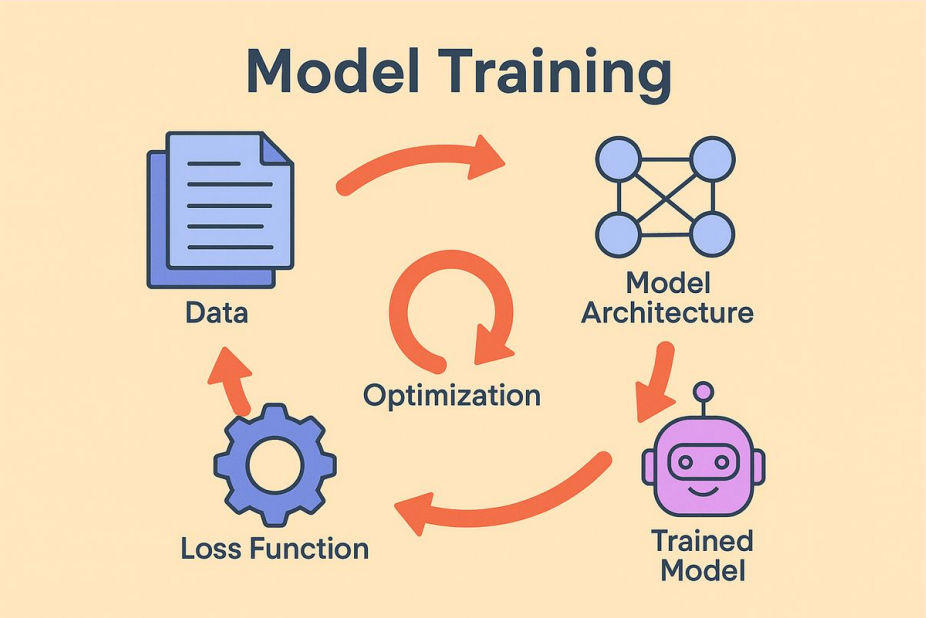
- Vision Transformer: Extracted via model.get_image_features().
- Text Transformer: Extracted via model.get_text_features().

Output: 512-dimensional vectors.

Core Metric: Cosine Similarity via Inner Product.



Model Weights & Training Data



Pretrained Weights: openai/clip-vit-base-patch32

Training Data: WebImageText (WIT) dataset.

Training Loop:

- Maximize dot-product for matching pairs.
- Minimize dot-product for incorrect pairs.

Fine-Tuning: Zero-shot transfer (No domain-specific retraining required).

End-to-End Architecture

Hardware Acceleration: Auto-detects cuda via torch.cuda.is_available(), falls back to cpu.

Phase 1: Offline Indexing (clip_faiss_search.ipynb)

- Ingest 500 images from torchvision.datasets.CIFAR10.
- Generate embeddings wrapped in torch.no_grad() to optimize memory.
- Persist image_embeddings.npy, image_paths.pkl, and faiss_index.bin.

Phase 2: Real-Time Application (app.py)

- Load cached artifacts (bypassing the 500-image encode pass).
- Encode live text queries and search the FAISS index.

Tensor Encoding & L2 Normalization

Vision Pipeline: PIL.Image -> CLIPprocessor(return_tensors="pt") -> Model Forward Pass.

Language Pipeline: String -> CLIPprocessor(padding=True) -> Model Forward Pass.

The Normalization Math:

- features = features / torch.norm(features, dim=-1, keepdim=True)

Application Interfacing: Tensors moved to .cpu().numpy().astype("float32") for FAISS compatibility.

The Vector Database (FAISS)

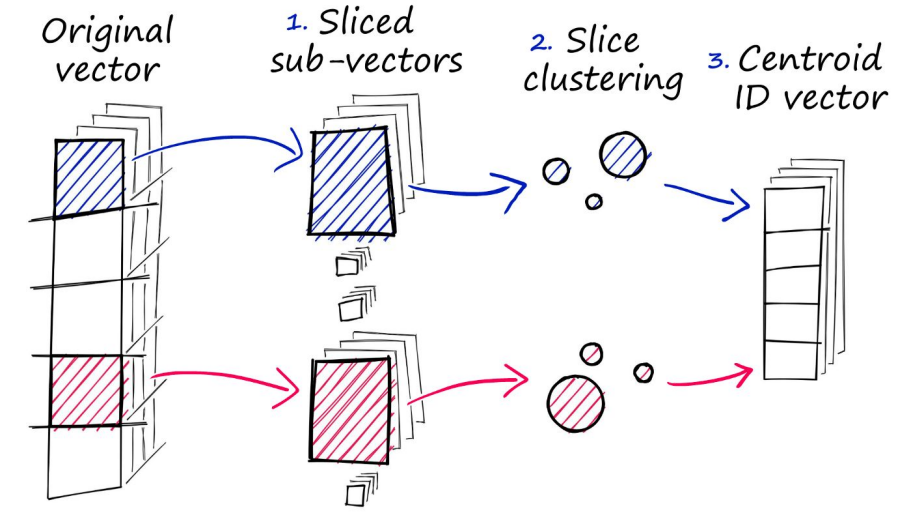
Technology: Facebook AI Similarity Search (FAISS).

Index Type: faiss.IndexFlatIP(dim) (Inner Product).

Search Execution: scores, idxs = index.search(vec, k)

Why FAISS?

- IndexFlatIP guarantees exact nearest-neighbour results.
- Decouples the storage/search layer from the neural network logic.



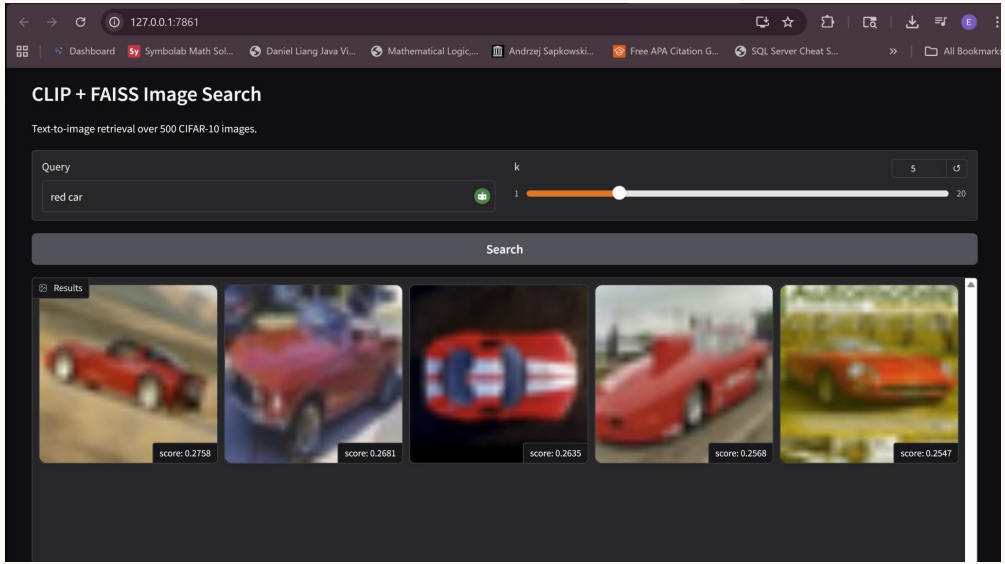
User Interface

UI Framework: Gradio gr.Blocks.

Components:

- gr.Textbox: Natural language input.
- gr.Slider: Adjust k (1 to 20, default 5).
- gr.Gallery: Renders the retrieved image paths and dynamically formatted scores.

Event Listeners: Bound to both search_btn.click and query_box.submit.



Technical Limitations & Dataset Constraints

CIFAR-10 Constraints:

- Images are strictly 32x32 pixels. Matches rely on coarse semantic shapes.

Architectural Limitations:

- Static Index: Appending a new image requires re-running the embedding loop and rebuilding faiss_index.bin.

- Compute Overhead: Every keystroke submission triggers a live CLIP forward pass; there is currently no text-query caching.

Scalability and Future Work

Scaling Vectors: Swap IndexFlatIP for IndexIVFFlat or IndexHNSWFlat to utilize Approximate Nearest Neighbours (ANN).

Future Optimizations:

- Implement a dictionary cache for text-query embeddings.
- Deploy embedding pipeline to batch cloud workers for larger, high-resolution datasets.